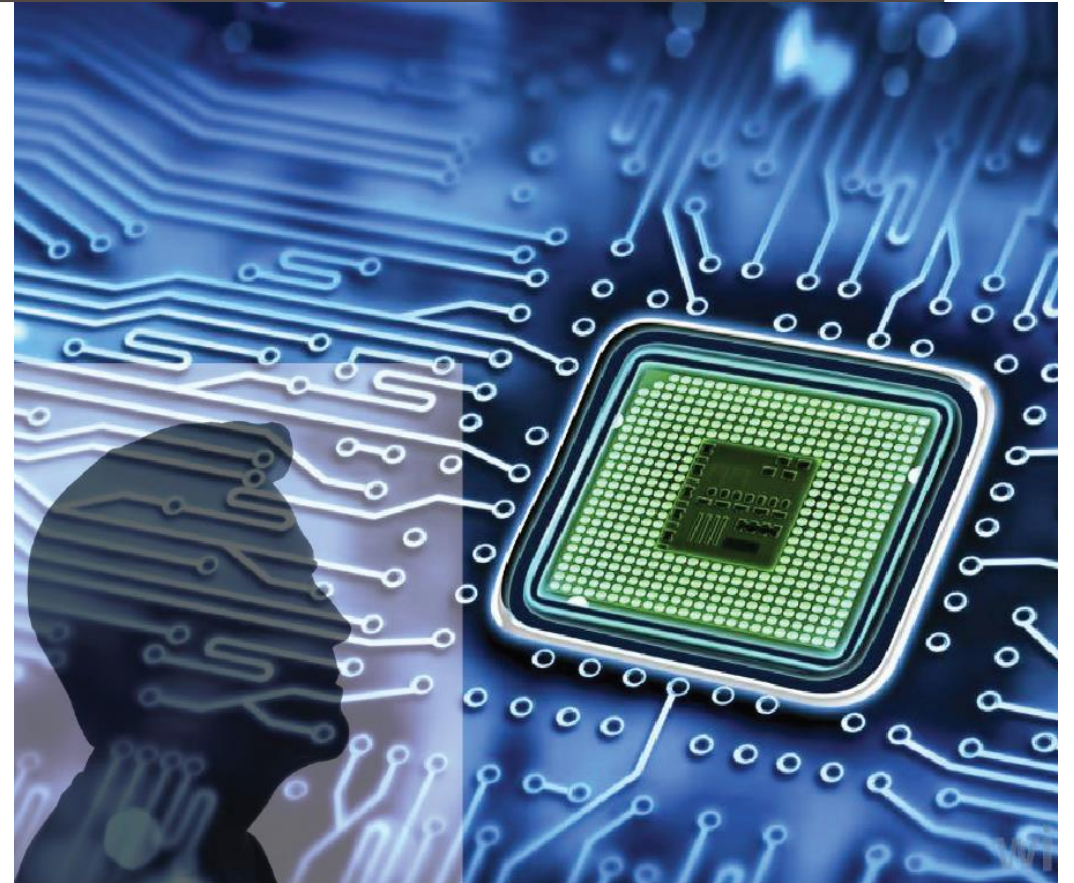


VLSI DESIGN FLOW

Week-7 Lecture-2 Formal Verification

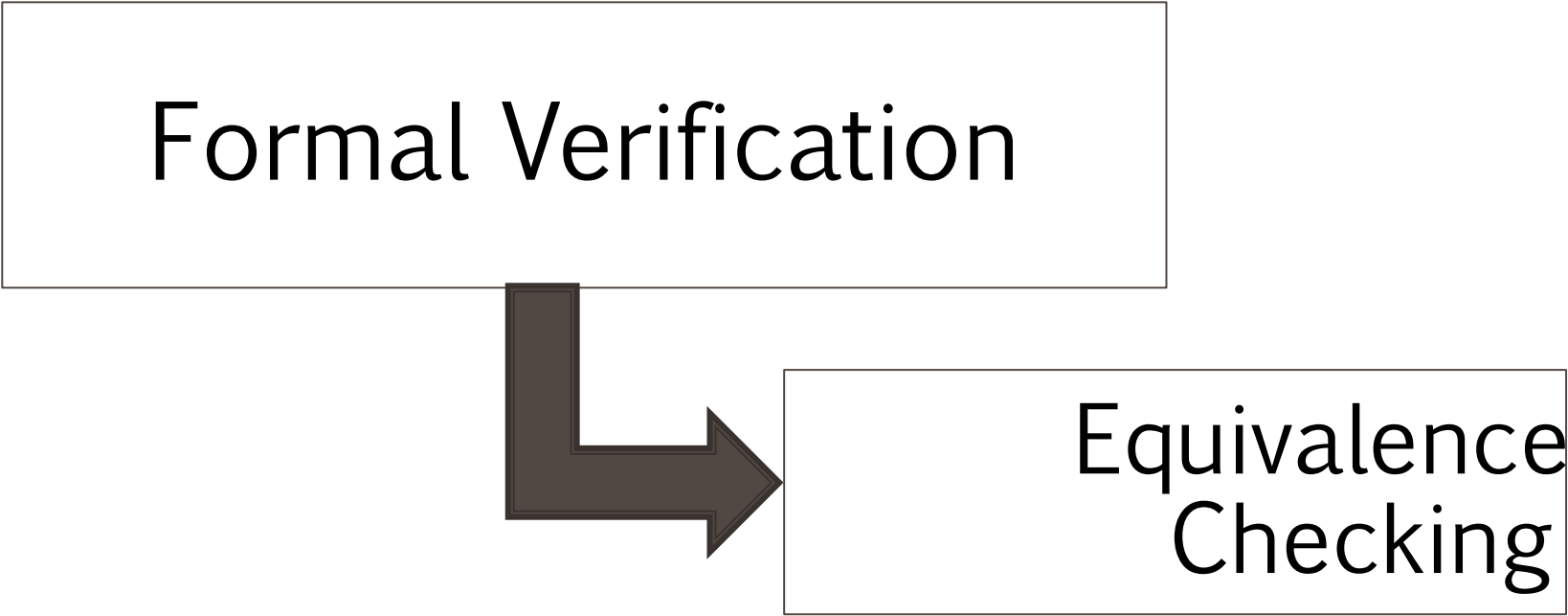
Sneh Saurabh
25th September, 2024



VDF: Revision/Announcements

-



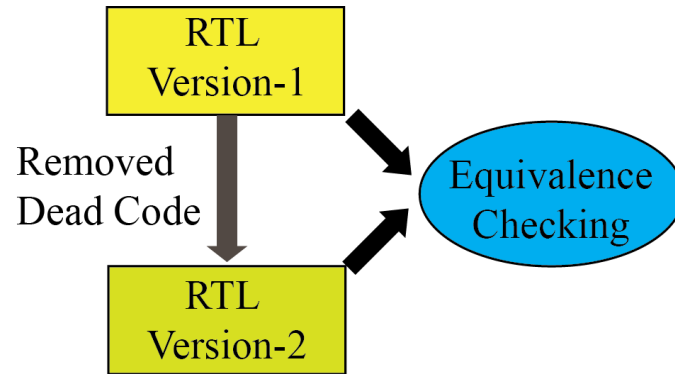


```
graph TD; A[Formal Verification] --> B[Equivalence Checking]
```

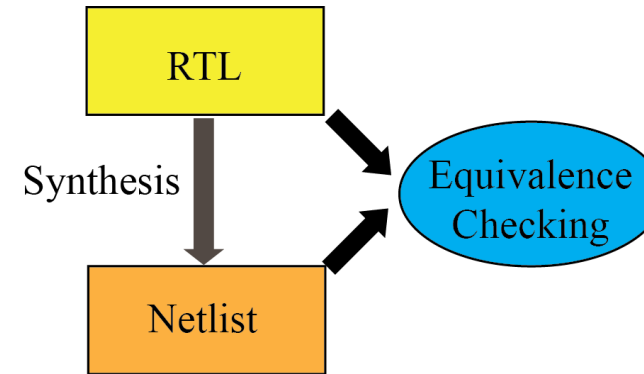
Formal Verification

Equivalence
Checking

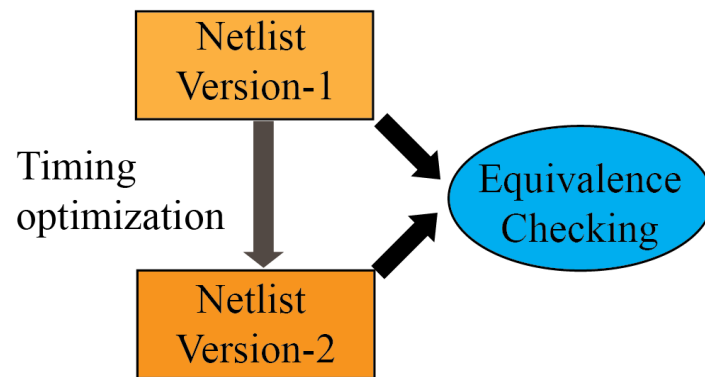
Equivalence Checking: Usage



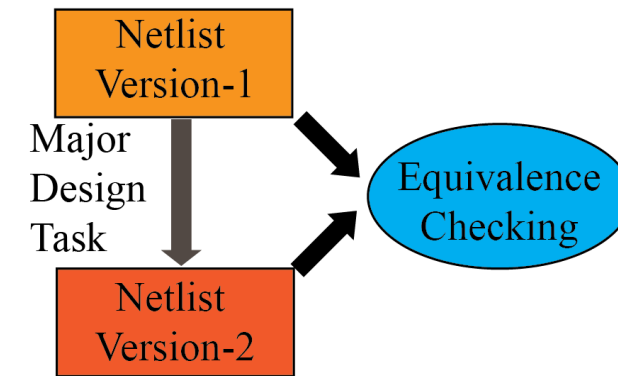
(a)



(b)

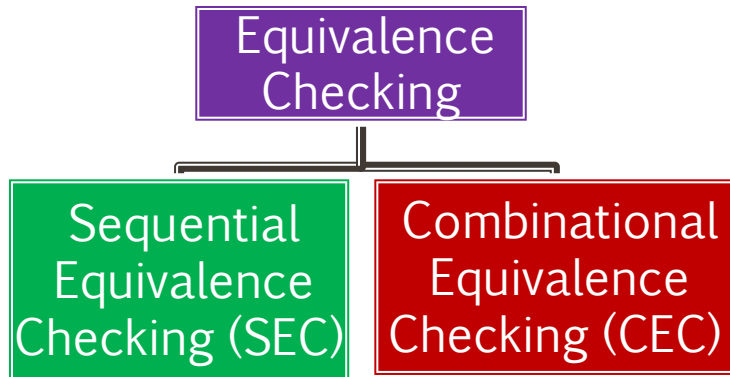


(c)



(d)

Formal Verification : Equivalence Checking (2)



Sequential Equivalence Checking

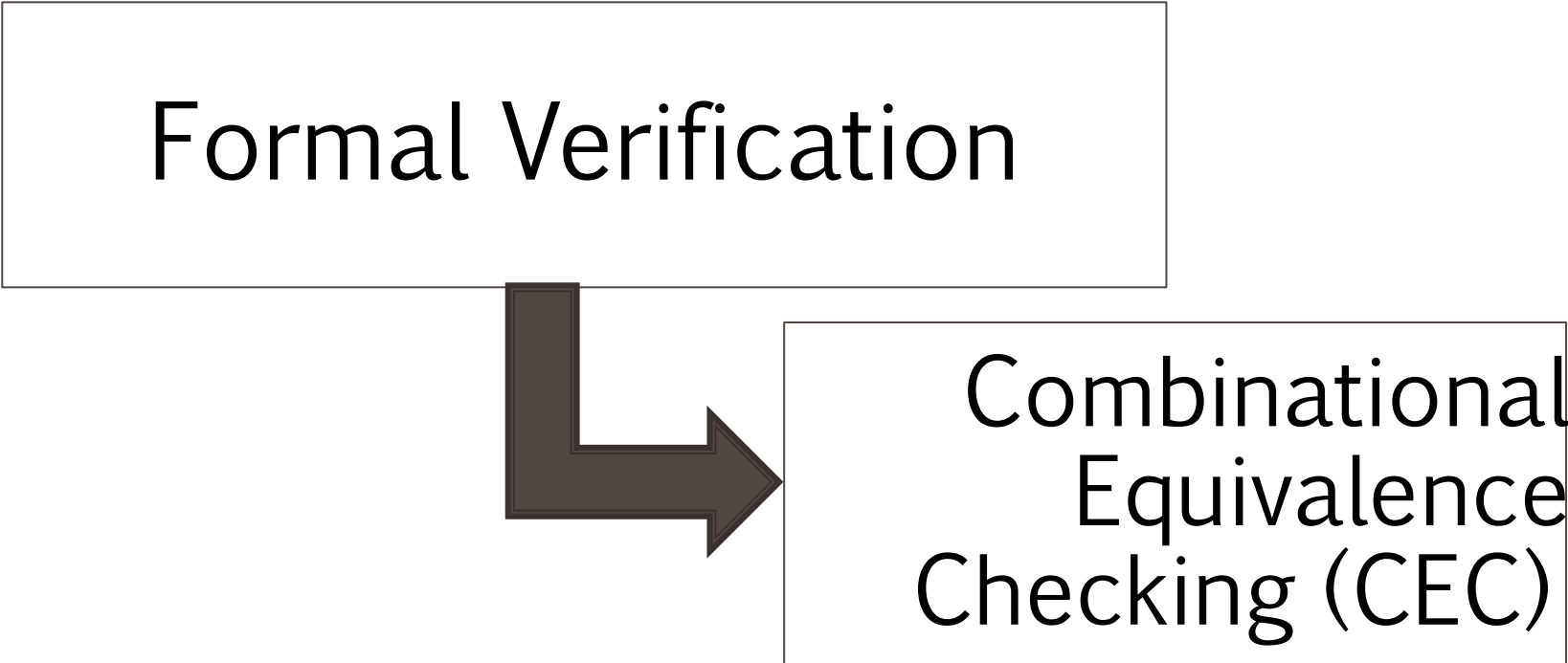
- Generalized approach to comparing two models
- First convert models to FSM
- Given their initial states, it is checked whether the two FSMs produce matching output sequences for all input sequences

Challenges of SEC:

- Need to explore all reachable states
 - Encounters state explosion problem
 - Computationally difficult

Combinational Equivalence Checking

- Assume that there is a one-to-one correspondence among memory elements or flip-flops and the ports of the two models.
- Problem reduced to establishing the equivalence of pairs of combinational circuits
- Assumption holds in most cases
 - Integral part of design flows

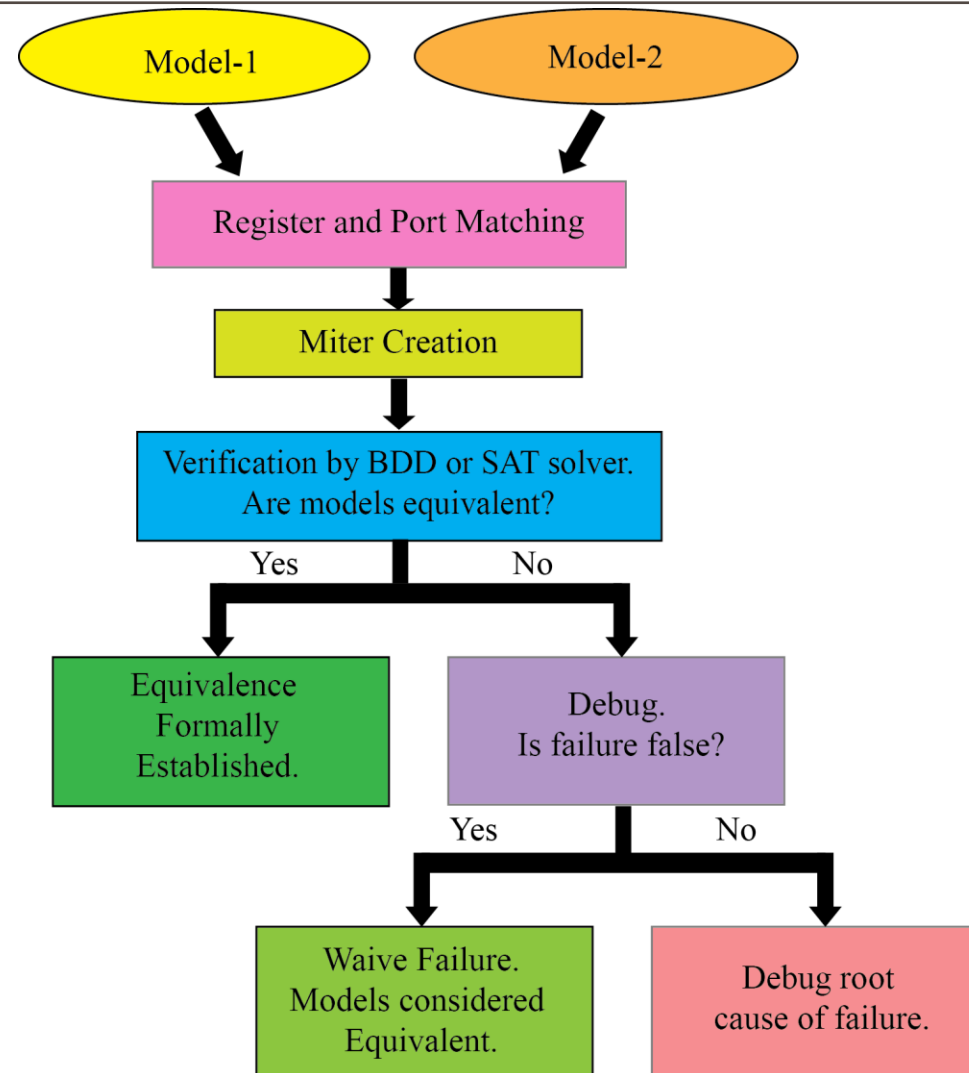


```
graph TD; A[Formal Verification] --> B[Combinational Equivalence Checking (CEC)]
```

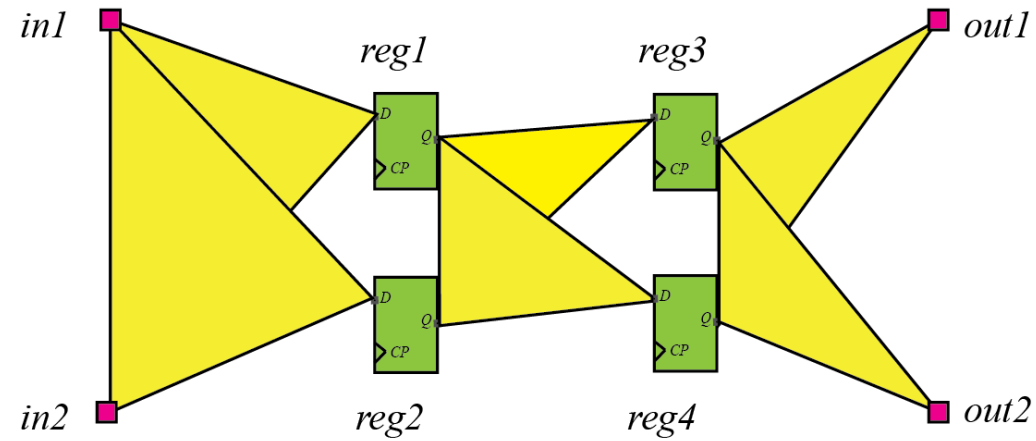
Formal Verification

Combinational
Equivalence
Checking (CEC)

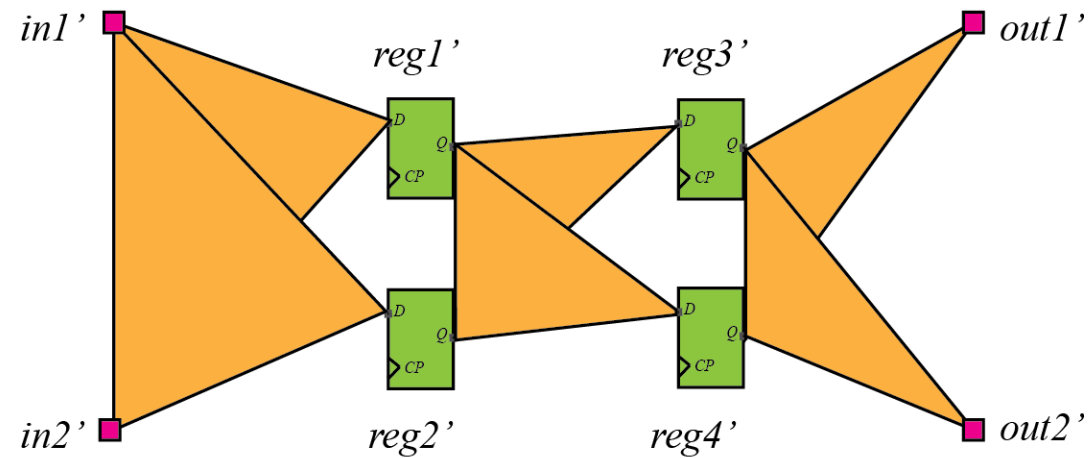
Combinational Equivalence Checking (CEC): Steps



CEC: Register and Port Matching (Illustration)



Model-1



Model-2

CEC: Register and Port Matching (Techniques)

No exact algorithm : heuristics employed

Name-based matching:

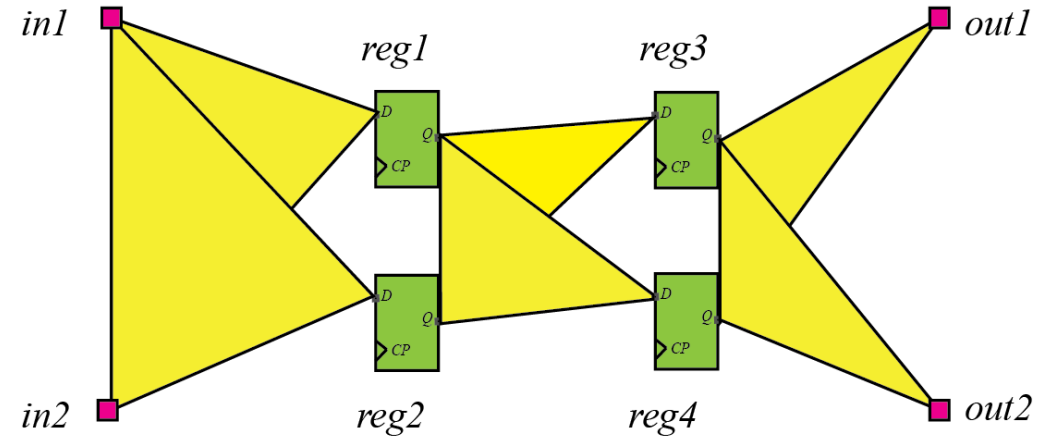
- Works very well for I/O Ports
- Works good where a model has not undergone sequential optimization and register correspondence is kept intact
- RTL synthesis keeps names of registers based on some rules/conventions (such as add suffix to the name of signal in the RTL code)

- Sophisticated structural or functional analysis can also be used
- User can guide the tools to match the registers

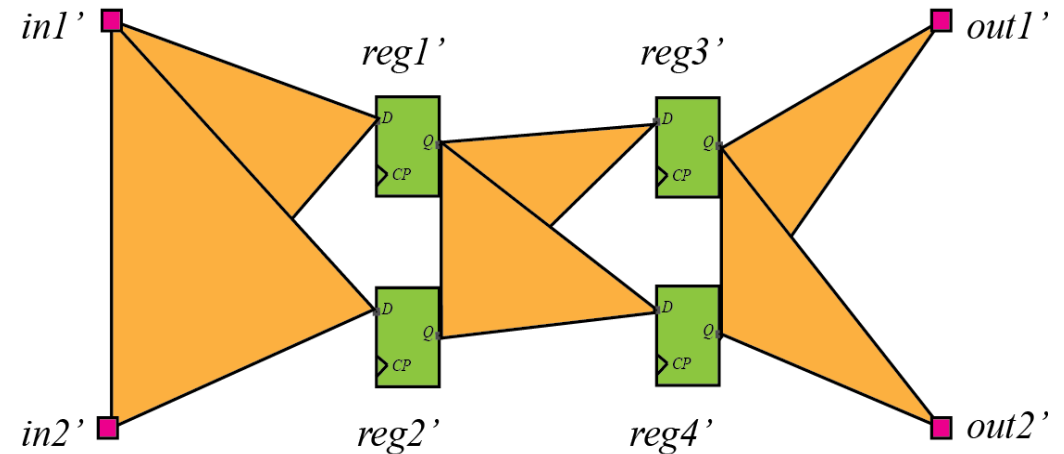
```
module flip_flop_d(d, clk, q);  
  input d, clk;  
  output q;  
  reg q;  
  
  always @(posedge clk) q <= d;  
endmodule
```

CEC: Miter Creation

- Need to ensure that the corresponding combinational circuits lying between matched registers/ports are equivalent
- Derive multiple small combinational circuits (known as mitters) from the given two models
 - A separate miter is derived for each register and output port of the two models

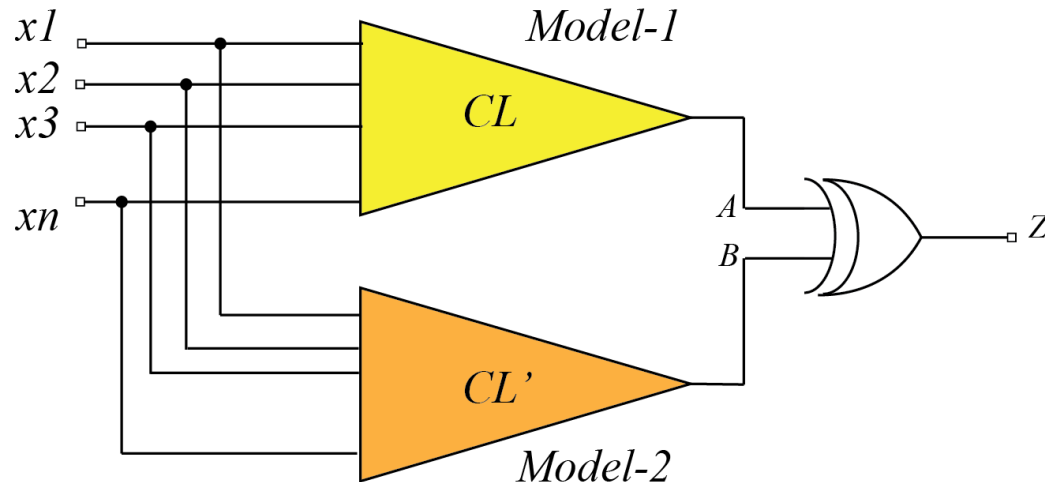
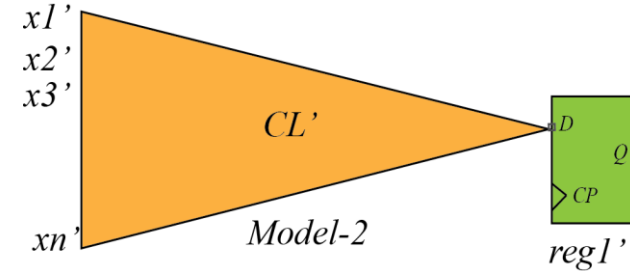
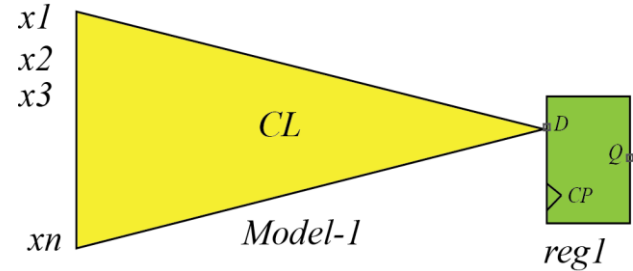


Model-1



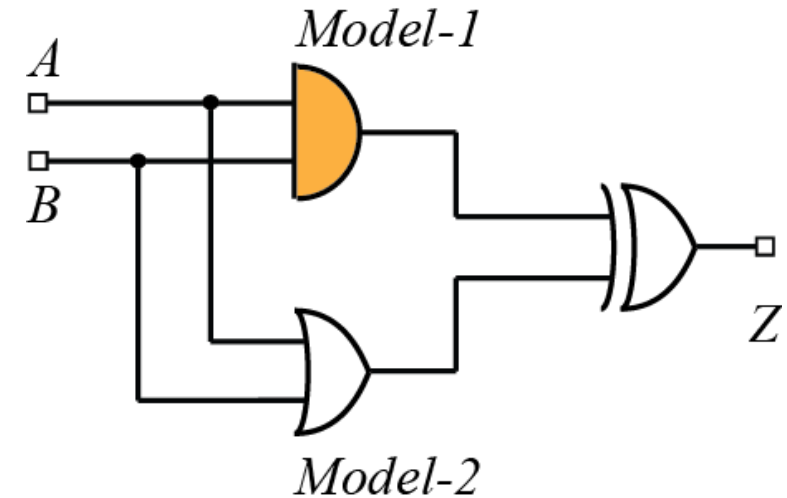
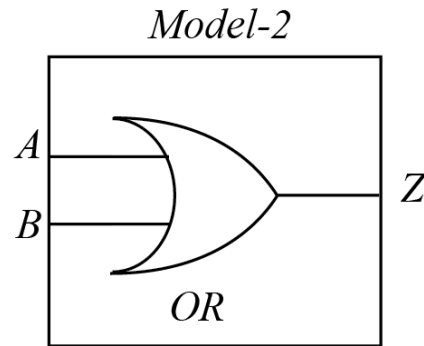
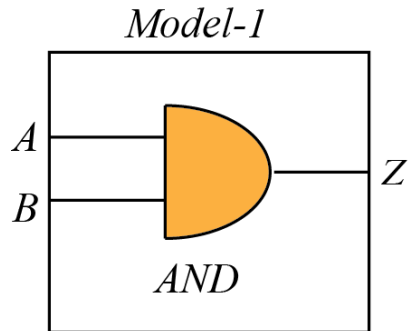
Model-2

CEC: Miter and Equivalence



- $Z=1$ for any combination of $\{x1, x2, x3, \dots, xn\} \Rightarrow$ Model-1 and Model-2 are not equivalent
- $Z=0$ for all possible combinations of $\{x1, x2, x3, \dots, xn\} \Rightarrow$ Model-1 and Model-2 are equivalent

CEC: Illustration



Given two models with ports matched by names.

- Let us draw the miter circuit for checking equivalence of Model-1 and Model-2.

- Let us determine patterns of values that can be assigned to ports A and B such that the output of the miter is 1.

- A=0, B=1
- A=1, B=0

CEC: Establishing Equivalence

Using BDDs:

- Build a BDD for a miter,
 - Equivalent models: reduce to a terminal vertex with the value of 0
 - Nonequivalent models: yield some other vertex.

Using SAT Solver:

- Invoke a SAT solver on the Boolean expression representing a miter
 - Equivalent models: UNSAT
 - Nonequivalent models: SAT

CEC:

- Create miter circuits for all flip-flops and output ports in the models
- For equivalent models, all miter circuits should be equivalent

Combinational Equivalence Checking: Soundness

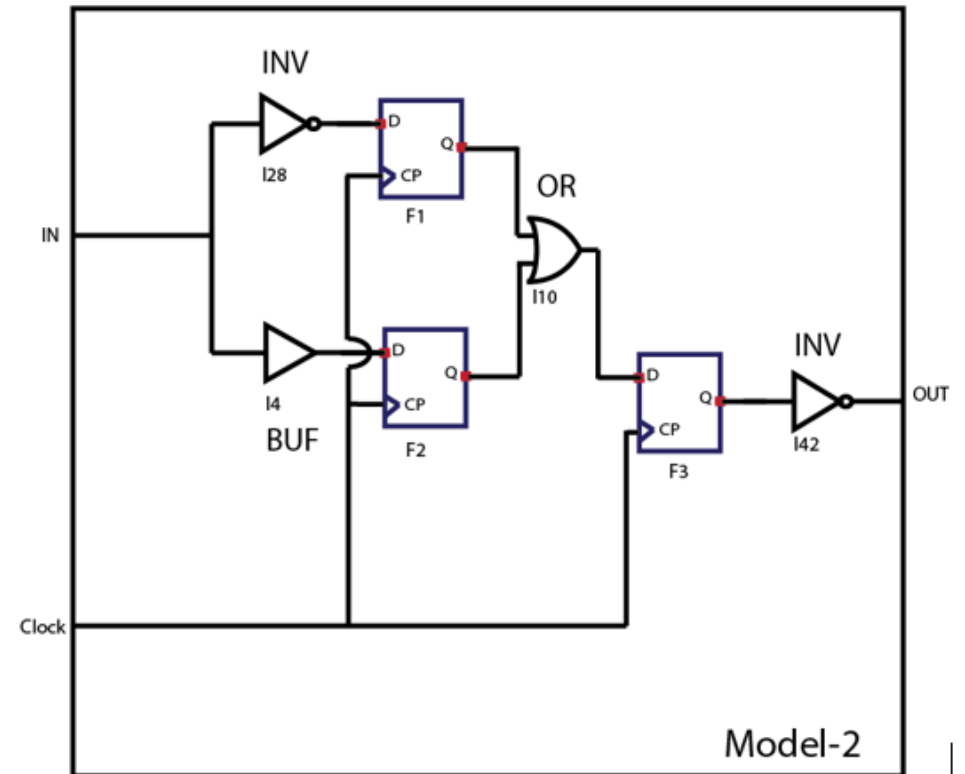
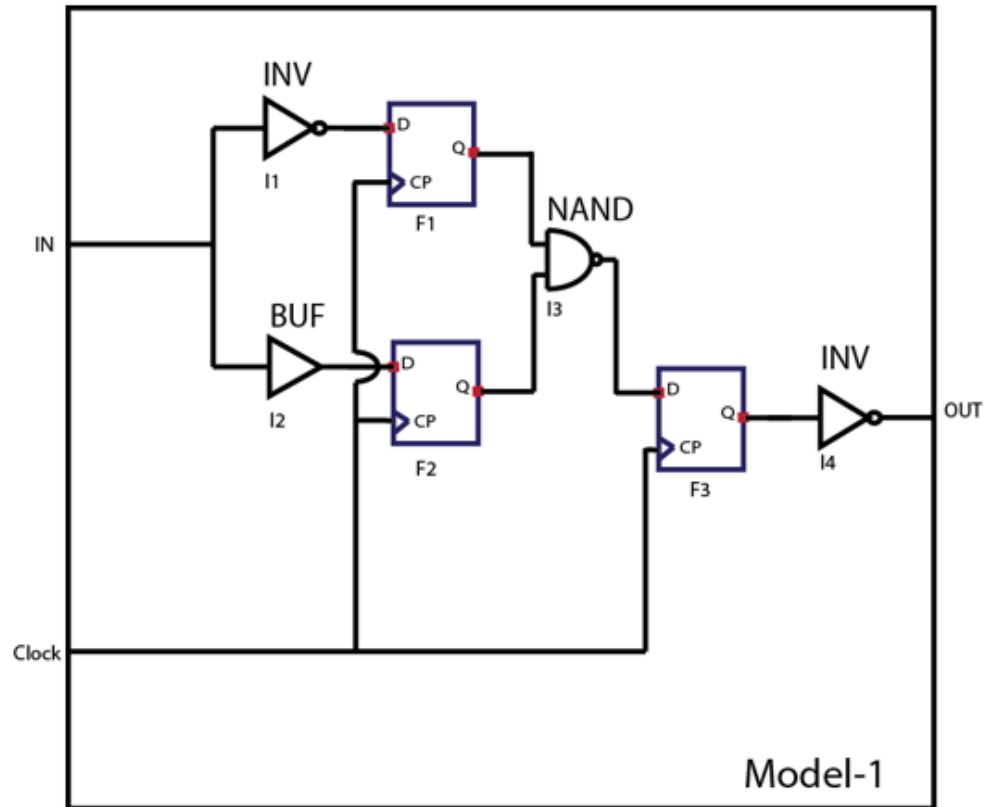
- CEC is a sound verification technique
 - It means that if CEC establishes equivalence, the two models are indeed functionally equivalent.
- If CEC shows non-equivalence, the two models can still be functionally equivalent.
 - The false non-equivalence can occur in some situations.

Example:

- Assume that a miter was found to be satisfiable for the input combination: $\{x1 = 1, x2 = 1, x3 = 1, x4 = 0\}$
- Input to a miter can be the output of registers (representing some state for an FSM)
 - The state $\{x1 = 1, x2 = 1, x3 = 1, x4 = 0\}$ may not be reachable (example, if the FSM is one-hot encoded)

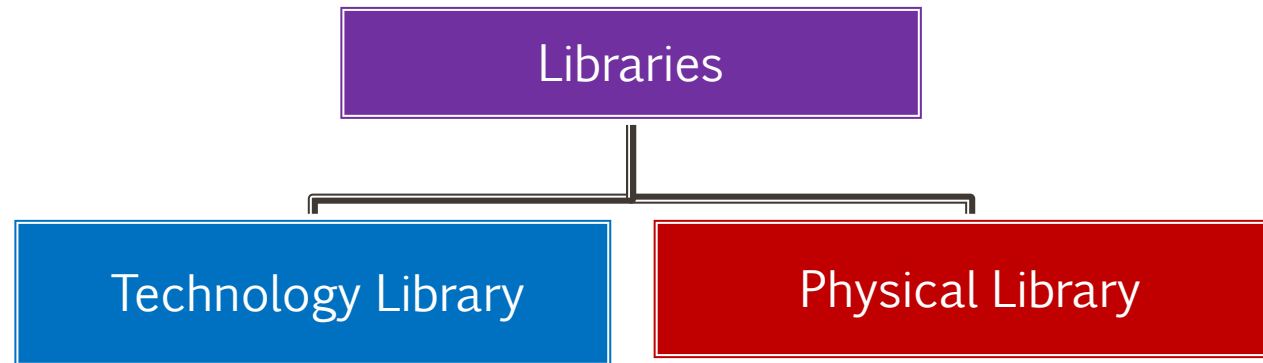
- CEC is more restrictive and safe
- False non-equivalence is not very common (failure can be debugged and waived)
- CEC is routinely employed in VLSI design flow

CEC: Problem



Technology Libraries

Libraries in VLSI Design Flow



Technology Library:

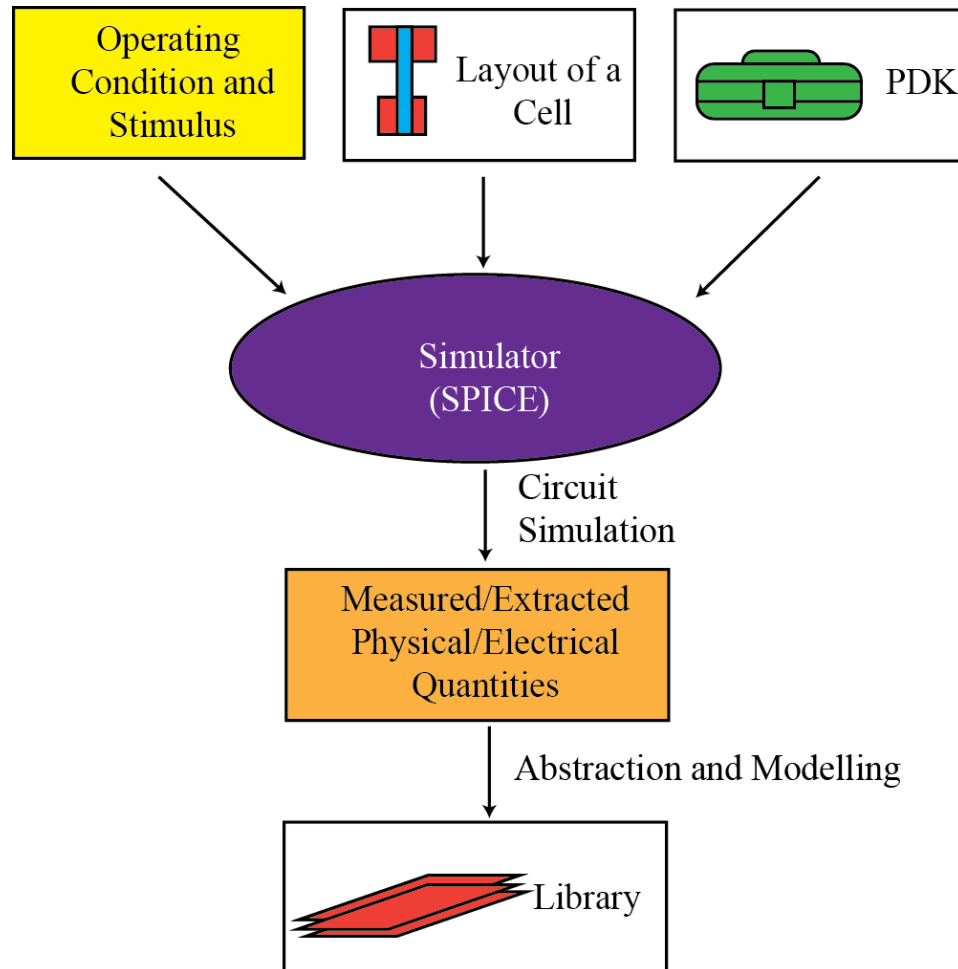
- Introduced for logic synthesis for defining the functionality of each cell.
- Evolved to support various design tasks such as timing verification, physical implementation, and test activities
- Also referred to as timing library.
- Liberty format
- ASCII files, typically with an extension of .lib.

Physical Library:

- Contains information about the geometry or layout of the cells, including design rules, in some abstract form.
- Library Exchange Format (LEF)

Library : How are libraries created? (1)

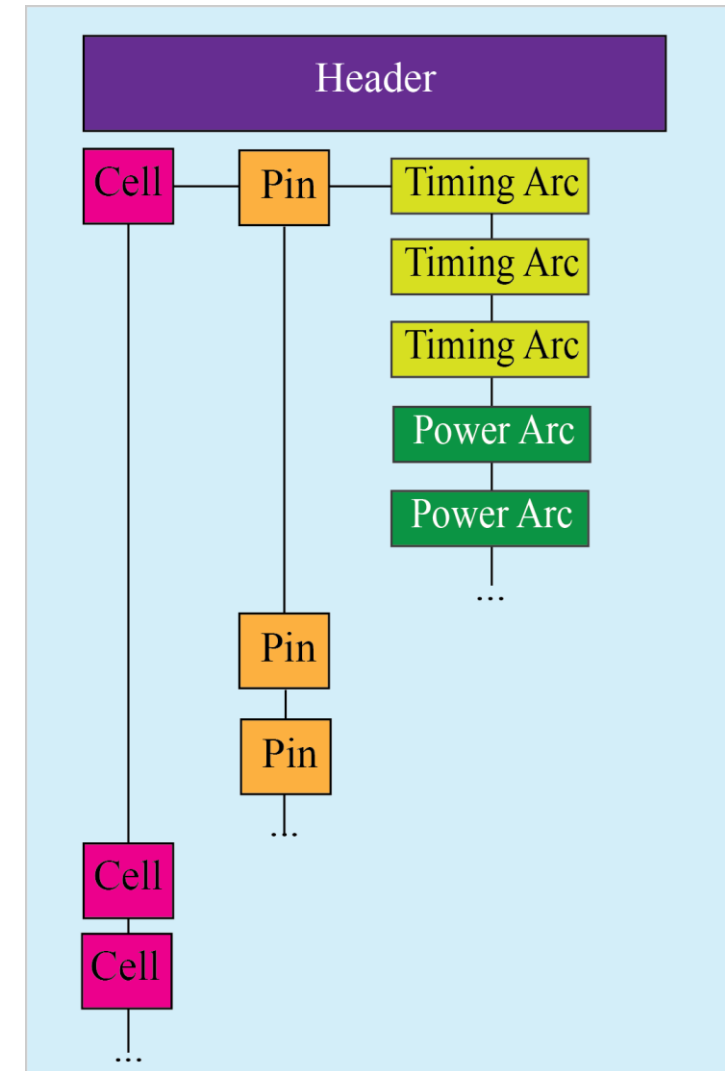
Library Characterization: process of creating the library (at foundry or design house)



- Design each cell optimally and verify
- SPICE simulations of each cells for:
 - Given operating condition and stimulus
 - Transistor model, process (retrieved from PDKs)
- Measure/extract the parameters of interest such as delay, slew, voltage, capacitance, power, etc.
- Build an abstract model and write in the given format

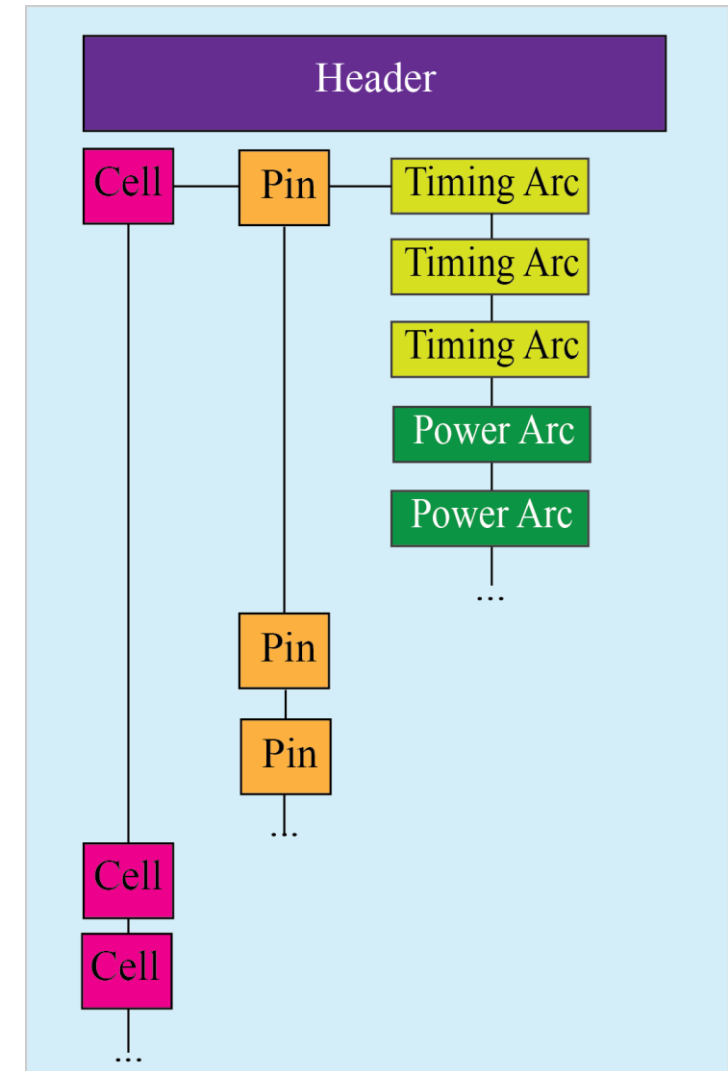
Library: Liberty format (1)

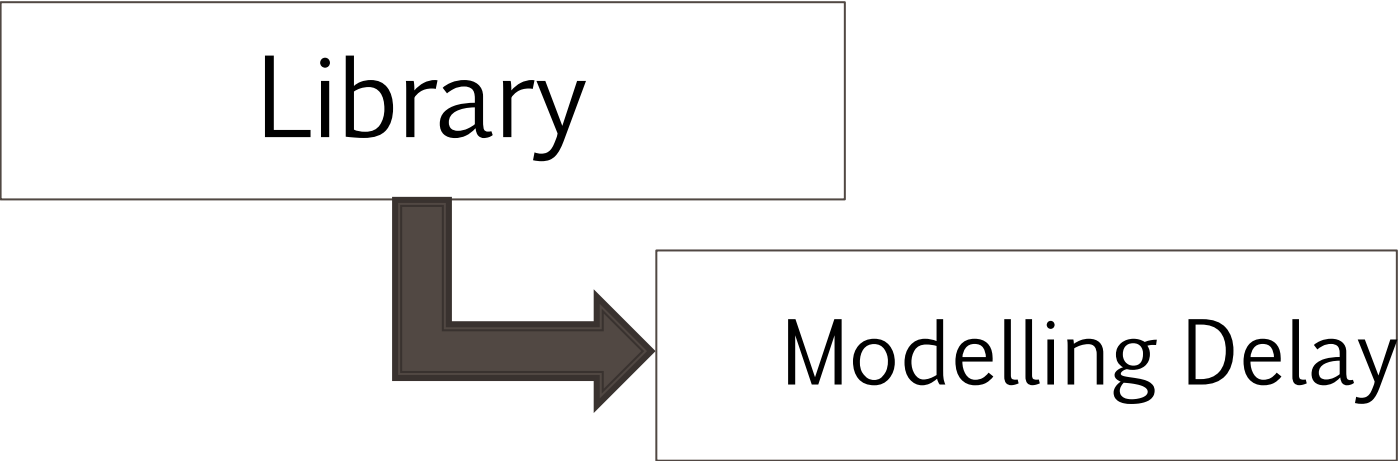
- Liberty format is simple ASCII/text format
- Data is primarily stored as attributes
 - Mapping between an attribute name and its value
 - Example: time_unit : “10ps”;
- Information is organized as a hierarchy of groups
- At the top level it has a Header
- Header contains:
 - PVT conditions, scaling factors, units
 - Information regarding characterizing conditions
 - Information that are valid for all the cells/pins/arcs
 - List of cells



Library: Liberty format (2)

- Cell contains:
 - Area
 - Cell Leakage Power
 - List of pins
- Pin contains:
 - Direction
 - Capacitance
 - Functionality (for output pins)
 - List of timing arcs
 - List of power arcs
- Timing arcs are used to perform timing analysis or computing delays of the arcs
- Power arcs are used to perform power analysis





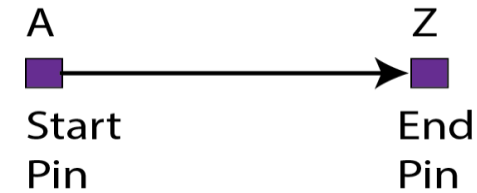
```
graph TD; Library[Library] --> ModellingDelay[Modelling Delay]
```

Library

Modelling Delay

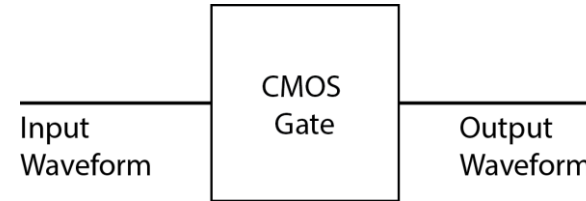
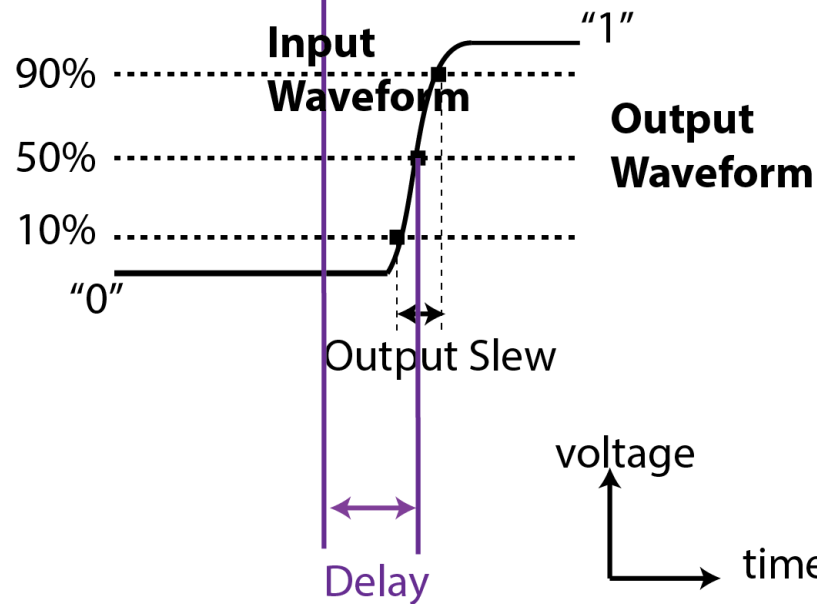
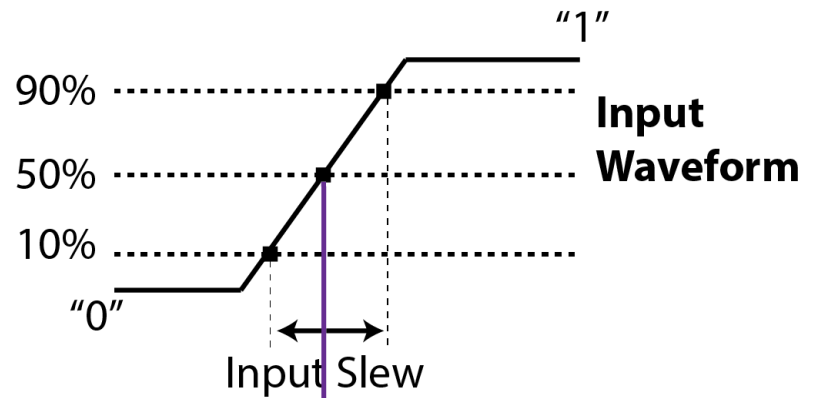
Library : Timing Arcs

- Timing Arcs are used to model *delay/transition* or *constraints* for combinational or sequential cells in a library
- Each Timing Arc has:
 - Start Pin : represented by attribute “related_pin” in Liberty syntax
 - End Pin : pin at which timing arc is defined in the library file
- Timing Arcs can be of type:
 - Delay
 - Constraints (Setup Check, Hold Check etc.)



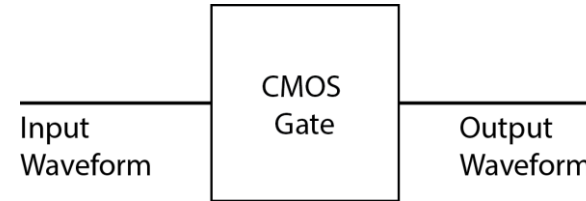
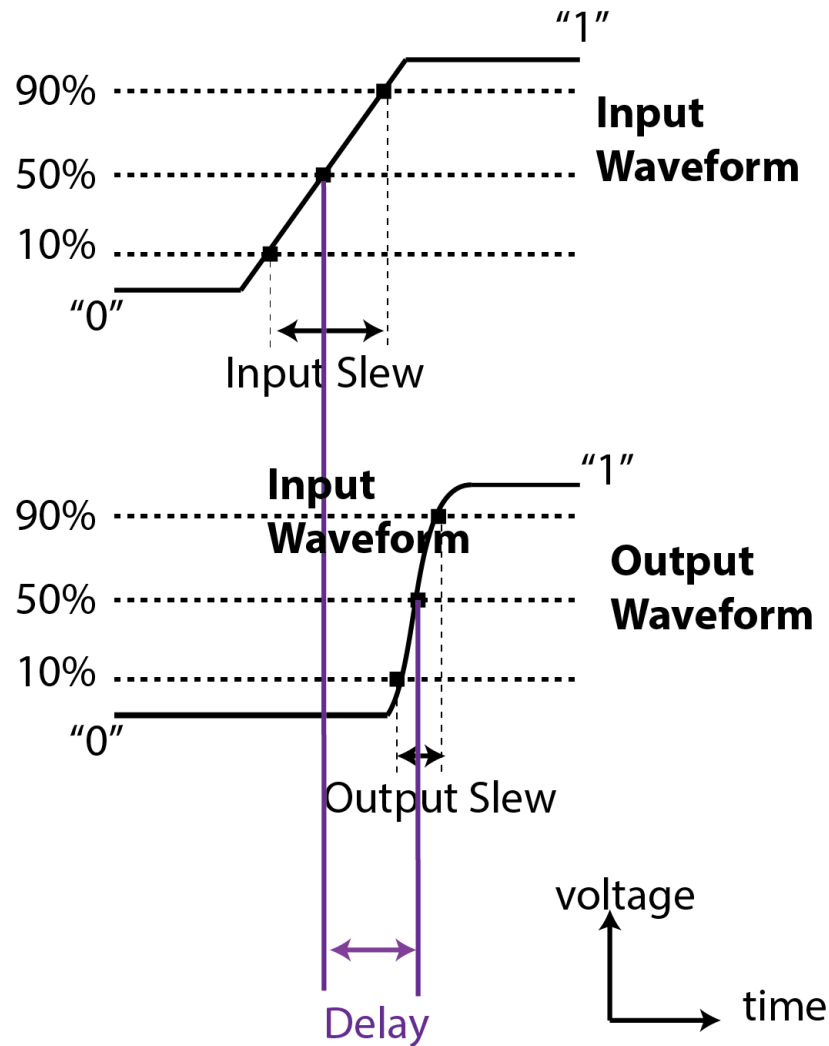
```
pin(Z) {  
    direction      : output ;  
    ...  
    timing() {  
        related_pin : "A" ;  
    }  
    ...  
}
```

CMOS Timing : Delay (Definition)



- Quantifies how much time it takes for the change in input to propagate to the output
- Measured as:
 - Time elapsed between the input signal reaching 50% of the supply voltage and the output signal reaching 50% of the supply voltage
- Also called: *rise delay, fall delay, propagation delay*

CMOS Timing : Slew (Definition)



- Quantifies how steeply or sharply transition occurs from "0" $\rightarrow$ "1" or "1" $\rightarrow$ "0"
- Measured as:
 - Rise transition: time taken for a signal to reach from 10% to 90% of supply voltage
 - Fall transition: time taken for a signal to reach from 90% to 10% of supply voltage
 - This is called 10-90 threshold
- Sometimes slew threshold of 30-70 can also be used
- Also called: *rise transition time, fall transition time, rise time, fall time*

References

- S. Saurabh, “Introduction to VLSI Design Flow”. Cambridge: *Cambridge University Press*, 2023.